

Deep Dives

API Gateway

Learn when and how to effectively incorporate API Gateways into your system design interviews.

What is an API Gateway?

There's a good chance you've interacted with an API Gateway today, even if you didn't realize it. They're a core component in modern architectures, especially with the rise of microservices.

Think of it as the front desk at a luxury hotel. Just as hotel guests don't need to know where the housekeeping office or maintenance room is located, clients shouldn't need to know about the internal structure of your microservices.

An API Gateway serves as a single entry point for all client requests, managing and routing them to appropriate backend services. Just as a hotel front desk handles check-ins, room assignments, and guest requests, an API Gateway manages centralized middleware like authentication, routing, and request handling.

The evolution of API Gateways parallels the rise of microservices architecture. As monolithic applications were broken down into smaller, specialized services, the need for a centralized point of control became evident. Without an API Gateway, clients would need to know about and communicate with multiple services directly – imagine hotel guests having to track down individual staff members for each request.

API gateways are thin, relatively simple components that serve a clear purpose. In this deep dive, we'll focus on what you need to know for system design interviews without overcomplicating things.

Core Responsibilities

The gateway's primary function is request routing – determining which backend service should handle each incoming request. But this isn't all they do.



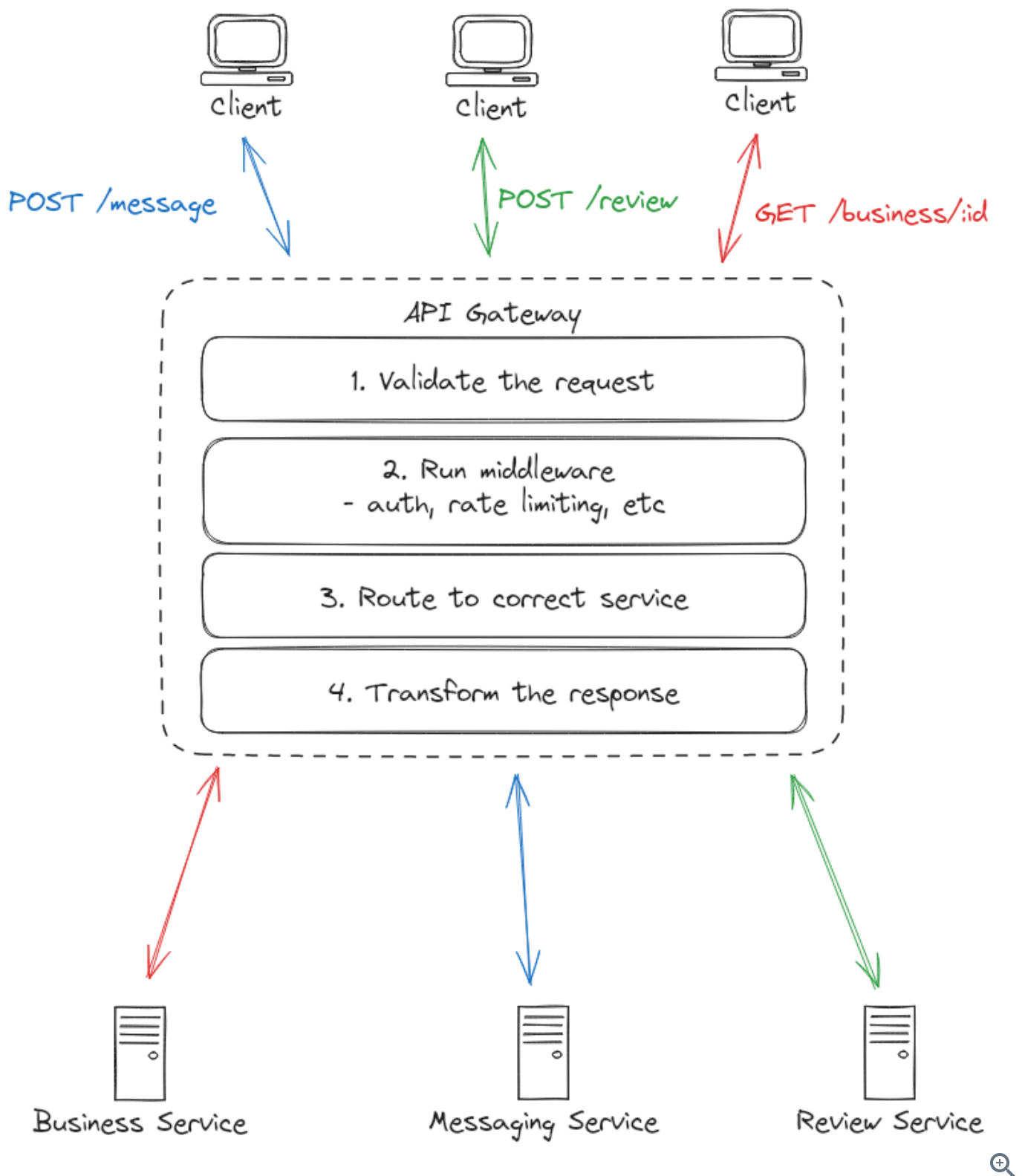
Funny enough, I'll often have candidates introduce a gateway in a system design interview and emphasize that it will do all this middleware stuff but never mention the core reason they need it -- request routing.

Nowadays, API gateways are also used to handle cross-cutting concerns or middleware like authentication, rate limiting, caching, SSL termination, and more.

Tracing a Request

Let's walk through a request from start to finish. Incoming requests come into the API Gateway from clients, usually via HTTP but they can be gRPC or any other protocol. From there, the gateway will apply any middleware you've configured and then route the request to the appropriate backend service.

1. Request validation
2. API Gateway applies middleware (auth, rate limiting, etc.)
3. API Gateway routes the request to the appropriate backend service
4. Backend service processes the request and returns a response
5. API Gateway transforms the response and returns it to the client
6. Optionally cache the response for future requests



Let's take a closer look at each step.

1) Request Validation

Before doing anything else, the API Gateway checks if incoming requests are properly formatted and contain all the required information. This validation includes checking that the request URL is valid, required headers are present, and the request body (if any) matches the expected format.

This early validation is important because it helps catch obvious issues before they reach your backend services. For example, if a mobile app sends a malformed JSON payload or forgets to include a required API key, there's no point in routing that request further into your system. The gateway can quickly reject it and send back a helpful error message, saving your backend services from wasting time and resources on requests that were never going to succeed.

2) Middleware

API Gateways can be configured to handle various middleware tasks. For example, you might want to:

- Authenticate requests using JWT tokens
- Limit request rates to prevent abuse
- Terminate SSL connections
- Log and monitor traffic
- Compress responses
- Handle CORS headers
- Whitelist/blacklist IPs
- Validate request sizes
- Handle response timeouts
- Version APIs
- Throttle traffic
- Integrate with service discovery

Of these, the most popular and relevant to system design interviews are authentication, rate limiting, and ip whitelisting/blacklisting. If you do opt to mention middleware, just make sure it's with a purpose and that you don't spend too much time here.



My suggestion when introducing a API Gateway to your design is to simply mention, "I'll add a API

Gateway to handle routing and basic middleware" and move on.

3) Routing

The gateway maintains a routing table that maps incoming requests to backend services. This mapping is typically based on a combination of:

- URL paths (e.g., `/users/*` routes to the user service)
- HTTP methods (e.g., `GET`, `POST`, etc.)
- Query parameters
- Request headers

For example, a simple routing configuration might look like:

```
routes:
  - path: /users/*
    service: user-service
    port: 8080
  - path: /orders/*
    service: order-service
    port: 8081
  - path: /payments/*
    service: payment-service
    port: 8082
```



The gateway will quickly look up which backend service to send the request to based on the path, method, query parameters, and headers and send the request onward accordingly.

4) Backend Communication

While most services communicate via HTTP, in some cases your backend services might use a different protocol like gRPC for internal communication. When this happens, the API Gateway can handle translating between protocols, though this is relatively uncommon in practice.

The gateway would, thus, transform the request into the appropriate protocol before sending it to the backend service. This is nice because it allows your services to use whatever protocol or format is most efficient without clients needing to know about it.

5) Response Transformation

The gateway will transform the response from the backend service into the format requested by the client. This transformation layer allows your internal services to use whatever protocol or format is most efficient, while presenting a clean, consistent API to clients.

For example:

```
// Client sends a HTTP GET request
GET /users/123/profile

// API Gateway transforms this into an internal gRPC call
userService.getProfile({ userId: "123" })

// Gateway transforms the gRPC response into JSON and returns it to the client over HTTP
{
  "userId": "123",
  "name": "John Doe",
  "email": "john@example.com"
}
```

6) Caching

Before sending the response back to the client, the gateway can optionally cache the response. This is useful for frequently accessed data that doesn't change often and, importantly, is not user specific. If your expectation is that a given API request will return the same result for a given input, caching it makes sense.

The API Gateway can implement various caching strategies too. For example:

1. **Full Response Caching:** Cache entire responses for frequently accessed endpoints
2. **Partial Caching:** Cache specific parts of responses that change infrequently
3. **Cache Invalidation:** Use TTL or event-based invalidation

In each case, you can either cache the response in memory or in a distributed cache like Redis.

Scaling an API Gateway

When discussing API Gateway scaling in interviews, there are two main dimensions to consider: handling increased load and managing global distribution.

Horizontal Scaling

The most straightforward approach to handling increased load is horizontal scaling. API Gateways are typically stateless, making them ideal candidates for horizontal scaling. You can add more gateway instances behind a load balancer to distribute incoming requests.



While API Gateways are primarily known for routing and middleware functionality, they often include load balancing capabilities. However, it's important to understand the distinction:

1. **Client-to-Gateway Load Balancing:** This is typically handled by a dedicated load balancer in front of your API Gateway instances (like AWS ELB or NGINX).
2. **Gateway-to-Service Load Balancing:** The API Gateway itself can perform load balancing across multiple instances of backend services.



This can typically be abstracted away during an interview. Drawing a single box to handle "API Gateway and Load Balancer" is usually sufficient. You don't want to get bogged down in the details of your entry points as they're more likely to be a distraction from the core functionality of your system.

Global Distribution

Another option that works well particularly for large applications with users spread across the globe is to deploy API Gateways closer to your users, similar to how you would deploy a CDN.

This typically involves:

1. **Regional Deployments:** Deploy gateway instances in multiple geographic regions
2. **DNS-based Routing:** Use GeoDNS to route users to the nearest gateway
3. **Configuration Synchronization:** Ensure routing rules and policies are consistent across regions

Popular API Gateways

Let's take a look at some of the most popular API Gateways.

Managed Services

Cloud providers offer fully managed API Gateway solutions that integrate well with their ecosystems. This is by far the easiest option but also the most expensive.

1. AWS API Gateway

- Seamless integration with AWS services
- Supports REST and WebSocket APIs
- Built-in features like:
 - Request throttling
 - API keys and usage plans
 - AWS Lambda integration
 - CloudWatch monitoring

2. Azure API Management

- Strong OAuth and OpenID Connect support
- Policy-based configuration
- Developer portal for API documentation

3. Google Cloud Endpoints

- Deep integration with GCP services
- Strong gRPC support
- Automatic OpenAPI documentation

Open Source Solutions

For teams wanting more control or running on-premises:

1. Kong

- Built on NGINX
- Extensive plugin ecosystem

- Supports both traditional and service mesh deployments

2. Tyk

- Native support for GraphQL
- Built-in API analytics
- Multi-data center capabilities

3. Express Gateway

- JavaScript/Node.js based
- Lightweight and developer-friendly
- Good for Node.js microservices

When to Propose an API Gateway

Ok cool, but when should you use an API Gateway in your interview?

The TLDR is: use it when you have a microservices architecture and don't use it when you have a simple client-server architecture.

With a microservices architecture, an API Gateway becomes almost essential. Without one, clients would need to know about and communicate with multiple services directly, leading to tighter coupling and more complex client code. The gateway provides a clean separation between your internal service architecture and your external API surface.

However, it's equally important to recognize when an API Gateway might be overkill. For simple monolithic applications or systems with a single client type, introducing an API Gateway adds unnecessary complexity.



I've mentioned this throughout, but I want it to be super clear. While it's important to understand every component you introduce into your design, the API Gateway is not the most interesting. There is a far greater chance that you are making a mistake by spending too much time on it than not enough.

Get it down, say it will handle routing and middleware, and move on.

Not sure where your gaps are?

Mock interview with an interviewer from your target company. Learn exactly what's standing in between you and your dream job.

[Schedule A Mock](#)

[Are Mocks Worth It?](#)

[Login To Join The Discussion](#)

Your account is free and you can post anonymously if you choose.

Sort By

Old



G GleamingChocolateChinchilla112

• 2 months ago • edited 28 days ago

Is it reasonable to use API Gateway as an aggregator layer? I never heard about it before, but I heard about the Backend-for-Frontend approach that does the same thing, but it introduces another microservice. Isn't this approach more common and flexible?

0



Evan King

• 2 months ago • edited 28 days ago

Both are valid but serve different purposes. API Gateway aggregation is simpler and has lower latency but less flexible. BFF offers more power for complex transformations and client-specific needs but adds complexity.

Most large scale systems would opt for BFF I'd think.

0



Andjela Arsic

• 2 months ago

I believe that this sentence is not finished :)

"Here's how you might approach this in different scenarios:"

0



Jagrit

• 2 months ago • edited 28 days ago

Great Article, In my interviews, whenever there is a microservices architecture which is the case 90% of time, First I discuss the problem as like we need to have traffic control or routing thing and then introduce api gateway. This way it is always easy to correlate the usecase and the solution. Also another good analogy I have read about apigw is that it is very similar to Road Traffic Police like as they manage the directions of cars, etc. Your hotel desk example is pretty cool as well!

Thanks once again!

👍 1



Evan King

• 2 months ago • edited 28 days ago

Spot on!

👍 1



Tushar Goyal

• 2 months ago • edited 28 days ago

I really liked the practicality of the article with end note that it's good to understand what is an API Gateway but there's no need to spend more than 30s on it in the interview. It's perfectly okay to say that we will need an API Gateway and write down the steps in brief it is going to perform such as routing, load balancing, rate limiting, etc. The interviewers almost never want to dive deep more on this topic.

👍 3



Evan King

• 2 months ago • edited 28 days ago

100%! Get it down and move on :)

👍 0



Ahmad

• 1 month ago

Awesome article. Learned a lot!

👍 0

sujon.sarkar31

• 1 month ago

I don't know if it is common in the industry. But in our company, our setup is below

client-> Api-gateway (rate-limits + user token generations) -> load-balancer ->service.

A single gateway for all microservices in the company. But each microservice has their own load balancer.

 0

F FeministTomatoMinnow506

• 1 month ago

It's a common practice. In Kubernetes, each service has a load balancer by default unless the configuration has been changed to use some other networking model.

 0

A

Alex Rivlin

• 1 month ago • edited 28 days ago

Today you will get a lot of JavaScript code that actually creates content needed for ML. What is your take on addressing this?

 0



Stefan Mai

• 1 month ago • edited 28 days ago

You mind elaborating on your question, Alex? Not quite sure what you mean.

 0

I

InterimBlueDove307

• 1 month ago • edited 28 days ago

Get it down, say it will handle routing and middleware, and move on.

Do I need to justify why I'm selecting AWS API Gateway (if I do)? Or can I trust that the interviewer just accepts my selection as a common industry choice and move on from there?

 0



Stefan Mai

• 1 month ago • edited 28 days ago

Very unlikely. It's less that this is a common choice and more that interviewers just don't care so much about this particular detail as they do the rest of the problem. Most bigtech employees don't actually know a lot about the details of the gateways they're clients of!

 0

N neerajmokha

• 1 month ago • edited 28 days ago

If we are to design a simple hotel website (say a very small scale mom and pop hotel) and not a part of a hotel chain, would we still include API gateway ? I believe micro services architecture is the right fit since it enables simpler scaling strategy (via horizontal) BUT at the same time, a monolithic makes most sense here to avoid complex code. So not sure if API gateway will make sense in this design or not. Thoughts ?

 0



Stefan Mai

• 1 month ago • edited 28 days ago

You just need a website for a mom and pop hotel. You don't need microservices, an API gateway, or anything else. Might slap a load balancer up front and have 2 redundant instances so you can deploy without downtime.

You do not need a microservice architecture for horizontal scaling - you can scale monoliths in exactly the same way.

 0

N neerajmokha

• 1 month ago • edited 28 days ago

Thanks Stefan. A follow up question here. Do we need a web server in this case ? To me, this is a static page that users see when they go the momandpophotel.com. Once they start putting dates, then application server can return the object and UI will be rendered from there. Does that seem correct ?

 0



Stefan Mai

• 1 month ago • edited 28 days ago

If you have bookings you need business logic so you're going to need a server. The line between application server and web server is often blurred, especially with mid-sized applications. Hello Interview hosts both our static assets and API under the same next.js instance.

 0



Izumrud

• 1 month ago • edited 28 days ago

I have more like a question or actually a "think out loud" topic.

So lets imagine we have the AWS mostly. Lets say we dont have the kubernetes and use for microservices the ECS. Honestly in my practice I super rarely have seen the API gateway even for ECS.

Usually ALB is just enough with configured listeners and its rules. I still don't understand in what cases we actually NEED the API Gateway? From what I google mostly for Lambda, which is kinda very limited on the logic, but for ECS? You can implement the AUTH on the application side of the load balancer, you can use WAF for ALB to use rate limiting. I mean, does it sound like API Gateway is ONLY for some limited and super simple structure of microservices?

👍 0



Stefan Mai

• 1 month ago • edited 28 days ago

You don't *have* to use anything! You could have single node boxes that ping between one another with custom auth and routing.

For Hello Interview we don't use an API gateway. We have a frontend web service that does most of these things and, where it needs, kicks requests to other services.

The reality is as you scale you'll notice you're (re)-inventing a lot of established patterns. Knowing about them helps you to realize "hey, we're on the way to reinventing an API gateway here, maybe we should just use a battle tested one." Keeping things simple early is a good idea (!) but knowing about patterns (and technologies that facilitate them) is both a way of growing as an engineer and demonstrating depth in an interview.

👍 1



Izumrud

• 1 month ago • edited 28 days ago

Great reply, thank you very much!

But just to clarify it from the System Design Interview - while drawing the schema, is it a better practice to introduce the API Gateway (again let's say with ECS) or is it better to introduce let's say only ALB and to say that we handle the auth within the apps, and rate limiting within the WAF and other things like validation on app level? I mean, from perspective of a "perfect solution" what will be considered a "more perfect"?

👍 0



Stefan Mai

• 1 month ago • edited 28 days ago

There's no such thing as a perfect solution. If you're covering your bases you're good. Most interviewers are used to people slapping a block labelled API GW/LB and focusing on more important aspects of the interview.

👍 0



Surinder Mehra

• 18 days ago • edited 18 days ago

I am little confused about tagging api gateway to microservices. why not for monolith hosting single rest api???

api gateway features listed above are like outsourcing cross cutting concerns(rate limiting, auth....) and focus on business of api. these benefits still applies to api hosted on monolith.

Routing can be handled by load balancers no ? AWS LB has path based/host based routing. ALB can additionally use auto-scalers to scale the targets based on input load or other criteria.

A monolith can be deployed on ECS and ALB along with autoscaling group ensures it is always running. Api-gateway on the other hand takes care of other concerns like authentication/authorisation , rate limiting etc.

Could you please shed some light on this.



0



Stefan Mai

• 18 days ago

You could use an API gateway for a monolith. But if you have a monolith, you can also just put all the API gateway functionalities in your monolith! It's oftentimes easier to do auth, logging, rate limiting, etc. in that single service (this is what we do for Hello Interview).

AWS' ALB has a lot of features of an API Gateway. They're overlapping concepts and the degree to which they overlap will vary by vendor — there's not a strict definition.



0



Dmitry Grigorenko

• 5 days ago

Request Validation seems questionable because it couples the service API with the API Gateway. An invalid request shall not introduce a sufficient load for the backend service (otherwise it's a vulnerability) and therefore I am not convinced there is a sound win for offloading validation to the API Gateway



0



Dmitry Grigorenko

• 5 days ago • edited 5 days ago

Since API Gateway shall have visibility into traffic, it has no choice but terminate SSL. However, there might be a regulatory requirement to maintain encryption in transit. So will then the Gateway create an own SSL connection with downstream services?



0



Stefan Mai

• 5 days ago

Yes. Many compliance requirements need TLS end to end.

 0

Schedule a mock interview

Meet with a FAANG senior+ engineer or manager and learn exactly what it takes to get the job.

[Schedule A Mock Interview](#)

Questions

[Meta SWE Interview Questions](#)

[Amazon SWE Interview Questions](#)

[Google SWE Interview Questions](#)

[Engineering Manager \(EM\) Interview Questions](#)

Compare Us

[Compare to Interviewing.io](#)

Links

[FAQ](#)

[Pricing](#)

[Become a Coach](#)

[Our Coaches](#)

[Learn System Design](#)
[Learn DSA](#)
[Guided Practice](#)
[Hello Interview Premium](#)

Legal

[Terms and Conditions](#)
[Privacy Policy](#)

Contact

support@hellointerview.com
7511 Greenwood Ave North
Unit #4238 Seattle, WA 98103